Mardi 03/11 — Les bases du CSS

Kevin Bardoux

mardi 3 novembre 2026

Table des matières

Objectifs 2
Matin — CSS : sélecteurs, cascade, box model, unités — exercices 3
- Rappel (5 min) 3
- Apport 1 — Une règle CSS et ses sélecteurs (15 min) 3
- Règle CSS — attention et vérification 7
- Exercice 1 — Sélecteurs (15 min) 7
  - Enregistrer (1 min) 8
- Apport 2 — Cascade et spécificité (15 min) 8
- Cascade — attention et vérification 10
- Exercice 2 — Cascade (15 min) 10
  - Enregistrer (1 min) 10
- Apport 3 — Le box model (10 min) 11
- Box model — attention et vérification 13
- Exercice 3 — Les cartes (15 min) 13
  - Enregistrer et envoyer (2 min) 15
- Les outils du navigateur (15 min) 15
- À retenir — ce matin 16
- Débrief 16

Après-midi — TP — CSS Diner, puis « CSS from scratch » 16
- Apport 5 — Une interface propre (10 min) 16
- Interface propre — attention et vérification 17
- Apport 6 — Les variables CSS (5 min) 18
- Variables — attention et vérification 19
- Exercice — Avant / après (30 min) 19

Enregistrer (1 min) 19
« CSS from scratch » (1 h 05) 20
Tutoriel — Mesurer un contraste dans le navigateur (10 min) 20
Enregistrer et envoyer (2 min) 23
Les éditeurs du navigateur 23
À retenir — cet après-midi 24
Critères de réussite 24
Si le temps le permet — La page perso, plus loin 24
Pour aller plus loin 24
Débrief 25
Sources 25

Note

Compétences visées : C3 · résolution de problème **Livrable** : `css-exercices` et `interface-propre` au vert, les captures avant / après de l'interface propre, et le `style.css` de la page personnelle.

! Le minimum du jour

- Les tests des exercices 1 à 3 de `css-exercices.test.js` au vert (le 4 se fait mercredi matin).
- `node --test interface-propre.test.js` au vert, avec les captures avant / après.
- Page perso : `node --test page-perso-style.test.js` au vert, ce soir ou jeudi en début d'après-midi.

Le reste (rubriques « Si le temps le permet » et encadrés « Défi ») est la suite naturelle, pour ceux qui avancent.

Objectifs

- Écrire une règle CSS et choisir le bon sélecteur.
- Prévoir quelle règle l'emporte en cas de conflit.
- Régler marges, bordures et espacements avec le box model.

- Appliquer les règles d'une interface propre, en variables.

Comment se passe la journée. Le matin, deux notions : la règle CSS (ses sélecteurs et la cascade) et le box model ; les unités sont l'apport de mercredi matin. L'après-midi, deux : l'interface propre et les variables CSS. Chacune suit le même ordre : pourquoi, définition, exemple, pas à pas, erreur fréquente, vérification ; un encadré « À retenir » termine chaque demi-journée.

Matin — CSS : sélecteurs, cascade, box model, unités — exercices

Rappel (5 min)

1. Quelle balise pour le contenu principal d'une page ?
2. Que fait `git restore` ?
3. Que renvoie `[1, 2, 3].length - 1` ?

Trois questions courtes, sans regarder vos notes : la veille, la semaine passée, puis plus loin. Répondez sur une feuille ; la correction suit, à l'oral.

Apport 1 — Une règle CSS et ses sélecteurs (15 min)

- **Pourquoi** : donner une apparence à la page, sans toucher au HTML.
- **Règle CSS** : un sélecteur, puis `propriété: valeur;`.

Pourquoi. Le HTML dit ce qu'est le contenu ; le CSS dit comment il s'affiche : couleurs, tailles, espacements. Séparer les deux permet de changer l'apparence de tout un site sans toucher aux pages.

Définition. Une règle CSS commence par un **sélecteur**, qui désigne les éléments visés, puis des **déclarations** : une propriété, deux-points, une valeur, un point-virgule. Les sélecteurs de base (MDN Web Docs, s. d.-c) : `p` (tous les paragraphes), `.important` (les éléments qui ont `class="important"`), `#intro` (l'élément qui a `id="intro"`, unique dans la page), `nav a` (les liens **situés dans** un `nav`).

Exemple. La couleur `#1f2a5a` s'écrit en hexadécimal : trois nombres, rouge `1f`, vert `2a`, bleu `5a`, les trois quantités de rouge, de vert et de bleu d'un pixel vues en semaine 2 ; on peut aussi l'écrire `rgb(31 42 90)` (MDN Web Docs, s. d.-f). Pour convertir à la main : la fiche Systèmes de numération.

```
h1 {
  color: #1f2a5a;
  font-size: 2rem;
}
.important { font-weight: bold; }
nav a { text-decoration: none; }
```

Résultat (avec un `nav` de deux liens, un titre, un paragraphe `class="important"` et un paragraphe sans classe) :



Qui est ciblé ? La même petite page, quatre fois ; à chaque fois, un sélecteur différent surligne les éléments qu'il vise :

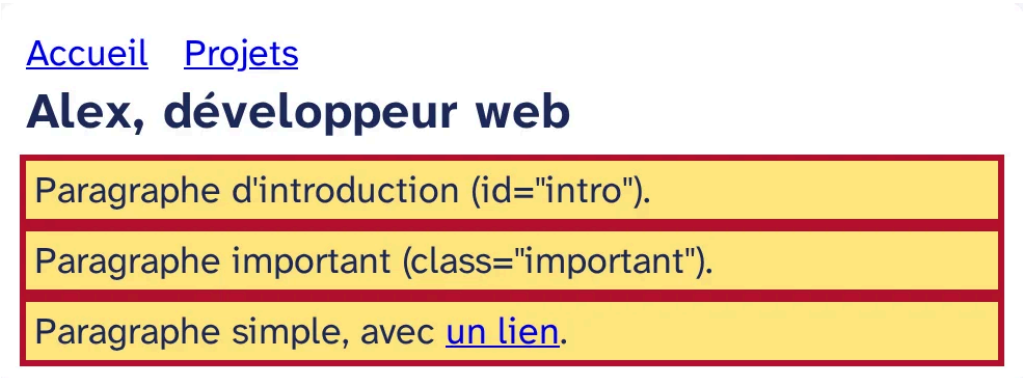


Figure 1. – `p { … }` : les trois paragraphes

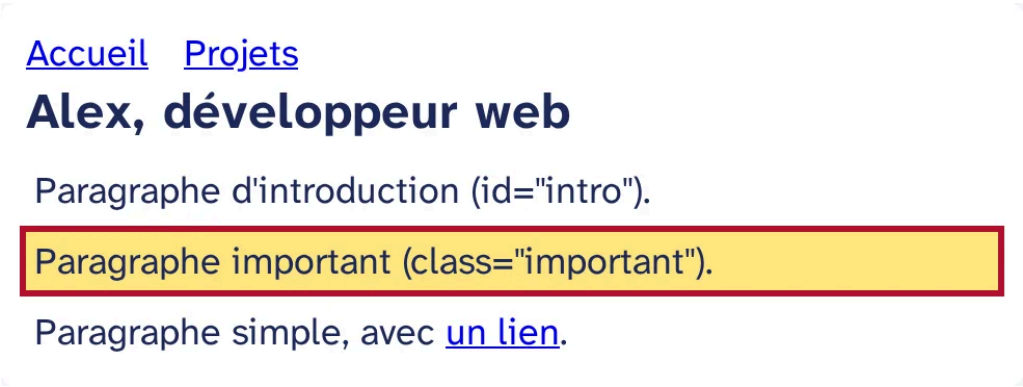


Figure 2. – `.important { … }` : le paragraphe important

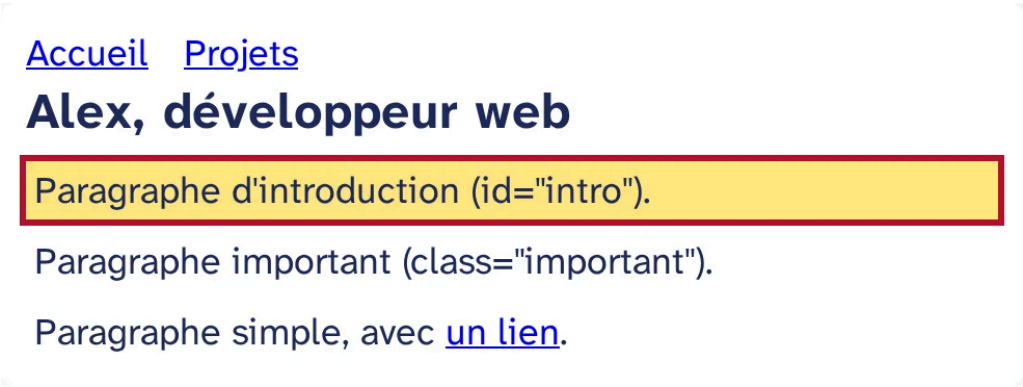


Figure 3. – `#intro { … }` : le paragraphe d'introduction



Figure 4. – `nav a { … }` : les deux liens du menu

Le code à gauche, le rendu à droite : chaque propriété de cet apport, seule, et ce qu'elle produit.

`color` : la couleur du texte

```
p { color: #b3122e; }
```

Alex, développeur web en formation.

`font-size` : la taille du texte

```css
p { font-size: 2rem; }
```



`font-weight` : la graisse

```css
p { font-weight: bold; }
```



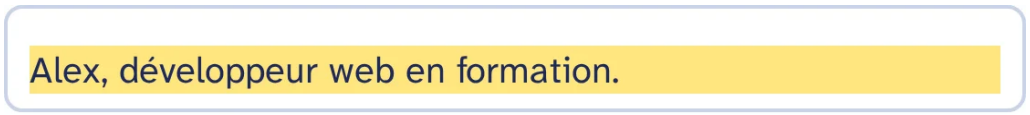
`text-decoration` : le soulignement

```css
a { text-decoration: none; }
```



`background` : le fond

```css
p { background: #ffe680; }
```



Pas à pas. Le formateur écrit chaque règle dans `css-exercices/style.css`, l'aperçu Live Preview ouvert à côté ; vous la recopiez aussitôt.

1. Écrivez la règle `h1`, enregistrez : le titre change de couleur dans l'aperçu.
2. Ajoutez `class="important"` à un paragraphe du HTML, puis la règle `.important` : seul ce paragraphe passe en gras.
3. Ajoutez la règle `nav a` : seuls les liens du menu perdent leur soulignement.

Règle CSS — attention et vérification

> ⚠ Attention
>
> Un point-virgule oublié : la déclaration et la **suivante** sont ignorées, sans message d'erreur. Si une propriété ne s'applique pas, regardez la ligne d'avant.

Avant / après : le point-virgule oublié après la couleur. Avant, ni la couleur ni la taille ne s'appliquent ; après, les deux.

Avant : le point-virgule oublié après la couleur

```
p {
  color: #b3122e
  font-size: 1.6rem;
}
```

Un paragraphe de la page.

Après : le point-virgule remis

```
p {
  color: #b3122e;
  font-size: 1.6rem;
}
```

Un paragraphe de la page.

- **Vérifiez** : quel sélecteur vise `<p class="note">` ?

Exercice 1 — Sélecteurs (15 min)

1. Ensemble (3 min) : la première consigne de l'exercice 1.
2. Seuls : les autres consignes, en commentaire dans `style.css`.
3. Vérifiez dans l'aperçu, puis `node --test css-exercices.test.js`.
4. **Réussi si** : les tests de l'exercice 1 passent.

Niveau : guidé.

Enregistrer (1 min)
Dans le terminal, depuis `~/dwwm/dwwm-tp/semaine-05` :

```
git status
git add css-exercices/style.css
git commit -m "Ajoute les règles de l'exercice 1 (sélecteurs)"
```

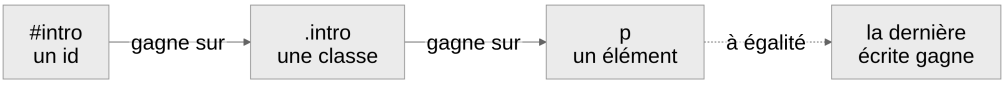
Réussi si : `git status` affiche `rien à valider, la copie de travail est propre`.

Apport 2 — Cascade et spécificité (15 min)

- **Pourquoi** : deux règles visent le même élément ; laquelle gagne ?
- **Spécificité** : un `id` bat une classe, qui bat un élément.

Pourquoi. Dans une vraie feuille de style, plusieurs règles touchent souvent le même élément. Sans savoir laquelle gagne, on ajoute des règles au hasard.

Définition. La cascade et la spécificité décident de la règle appliquée en cas de conflit (MDN Web Docs, s. d.-a) : d'abord la spécificité (un `id` bat une classe, qui bat un élément) (MDN Web Docs, s. d.-d) ; à spécificité égale, la **dernière** règle écrite l'emporte.



Qui gagne quand deux règles visent le même élément : d'abord la plus spécifique ; à égalité, la dernière écrite.

Le score de spécificité. On compte, dans l'ordre, les `id`, les classes et les éléments du sélecteur ; on compare les scores de gauche à droite :

Sélecteur	id	classe	élément	Il compte
`p`	0	0	**1**	un élément
`nav a`	0	0	**2**	deux éléments
`.bouton`	0	**1**	0	une classe
`p.important`	0	**1**	**1**	une classe et un élément

Sélecteur	id	classe	élément	Il compte
`#intro`	**1**	0	0	un id

On compare de gauche à droite : 1-0-0 bat 0-1-1, qui bat 0-1-0. À égalité, la dernière règle écrite gagne.

Exemple.

```
button { background: grey; }
.bouton { background: crimson; }   /* gagne : une classe */
button { background: navy; }       /* perd, même écrite après */
```

Pas à pas.

1. Ouvrez les outils du navigateur (F12) et cliquez sur le bouton.
2. Dans l'onglet des styles, la règle gagnante est en haut ; les déclarations perdantes sont **barrées**.
3. Déplacez la règle `.bouton` tout en bas, puis tout en haut : elle gagne toujours.

Quelle règle gagne ? L'exemple, étape par étape, avec le rendu réel de chaque étape :

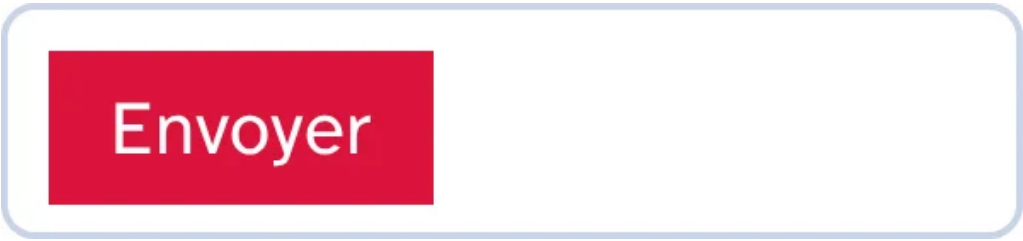
Étape 1 : une seule règle, le bouton est gris

```
button { background: grey; }
```



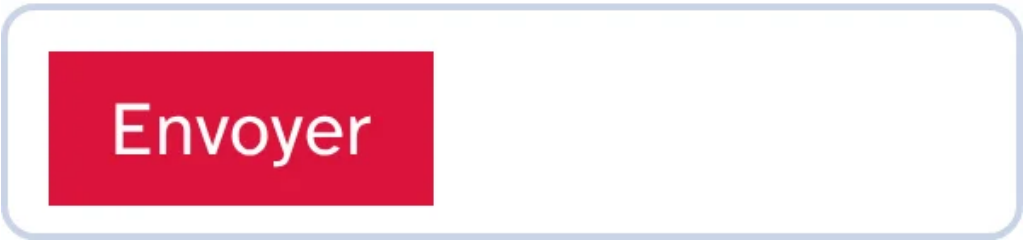
Étape 2 : la classe gagne sur l'élément

```
button { background: grey; }
.bouton { background: crimson; }
```



Étape 3 : écrite après, mais moins spécifique, la dernière règle perd

```
button { background: grey; }
.bouton { background: crimson; }
button { background: navy; }
```



Cascade — attention et vérification

> ⚠ Attention
>
> « Ma règle ne s'applique pas » : ouvrez F12 et cherchez-la barrée. Une règle plus spécifique gagne, même écrite avant.

- **Vérifiez** : `#intro` ou `.intro` : lequel gagne ?

Exercice 2 — Cascade (15 min)

1. Seuls : la consigne de l'exercice 2 dans `style.css`.
2. Déplacez votre règle avant, puis après la règle `button`.
3. **Réussi si** : vous expliquez par écrit ce qui se passe, et pourquoi.

Niveau : guidé.

Enregistrer (1 min)

Dans le terminal, depuis `~/dwwm/dwwm-tp/semaine-05` :

```
git status
git add css-exercices/style.css
```

```
git commit -m "Ajoute les règles de l'exercice 2 (cascade)"
```

Réussi si : `git status` affiche `rien à valider, la copie de travail est propre`.

Apport 3 — Le box model (10 min)

- **Pourquoi** : régler les espaces autour et à l'intérieur des éléments.
- **Box model** : contenu, `padding`, `border`, `margin`.

Pourquoi. Deux cartes collées, un texte qui touche le bord : la plupart des problèmes de mise en forme sont des problèmes d'espace.

Définition. Chaque élément est une boîte, le box model : le contenu, la marge intérieure (`padding`), la bordure (`border`), la marge extérieure (`margin`) (MDN Web Docs, s. d.-b). Avec `box-sizing: border-box`, la largeur annoncée inclut `padding` et `border`.

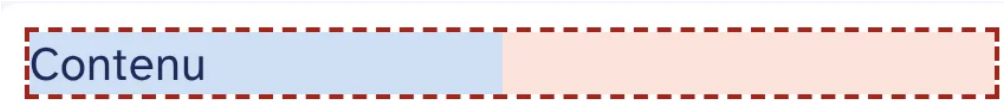


Figure 5. – Le box model : le contenu, puis `padding`, `border` et `margin`. Avec `border-box`, la largeur compte le contenu, le `padding` et la `border`.

Couche par couche : la même carte, en ajoutant une propriété à chaque fois ; le parent, en pointillés, montre la place prise par la `margin`.

1. Le contenu

```
.carte {
  width: 200px;
}
```



2. + `padding`

```
.carte {
  width: 200px;
  padding: 20px;
}
```



3. + `border`

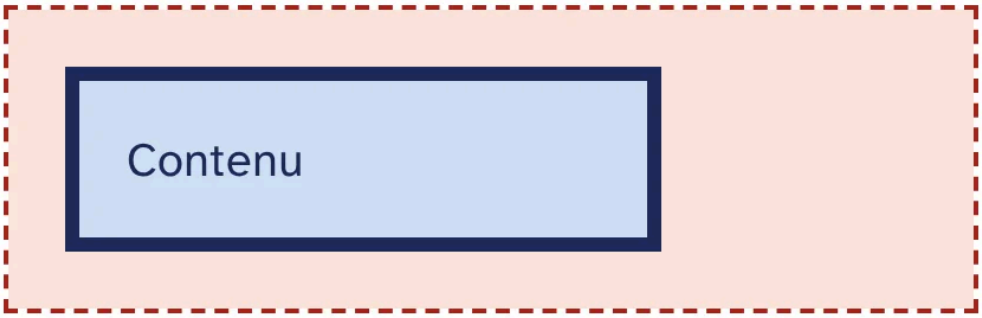
```
.carte {
  width: 200px;
  padding: 20px;
  border: 6px solid #1f2a5a;
}
```



4. + `margin`

```
.carte {
  width: 200px;
  padding: 20px;
```

```
  border: 6px solid #1f2a5a;
  margin: 24px;
}
```



Exemple.

```
*, *::before, *::after {
  box-sizing: border-box;
}
.carte { padding: 1rem; border: 1px solid #ccc; margin:
1rem 0; }
```

Pas à pas.

1. F12, puis cliquez sur une carte.
2. Dans l'onglet des styles calculés, regardez le schéma : contenu au centre, puis `padding`, `border`, `margin`.
3. Survolez chaque couche du schéma : la zone correspondante se colore dans la page.

Box model — attention et vérification

> ⚠ Attention
>
> Sans `border-box`, une boîte de `width: 300px` avec `padding: 20px` mesure 340 px : elle déborde. Mettez la règle `border-box` en haut de chaque feuille de style.

- **Vérifiez** : une marge **intérieure** : `padding` ou `margin` ?

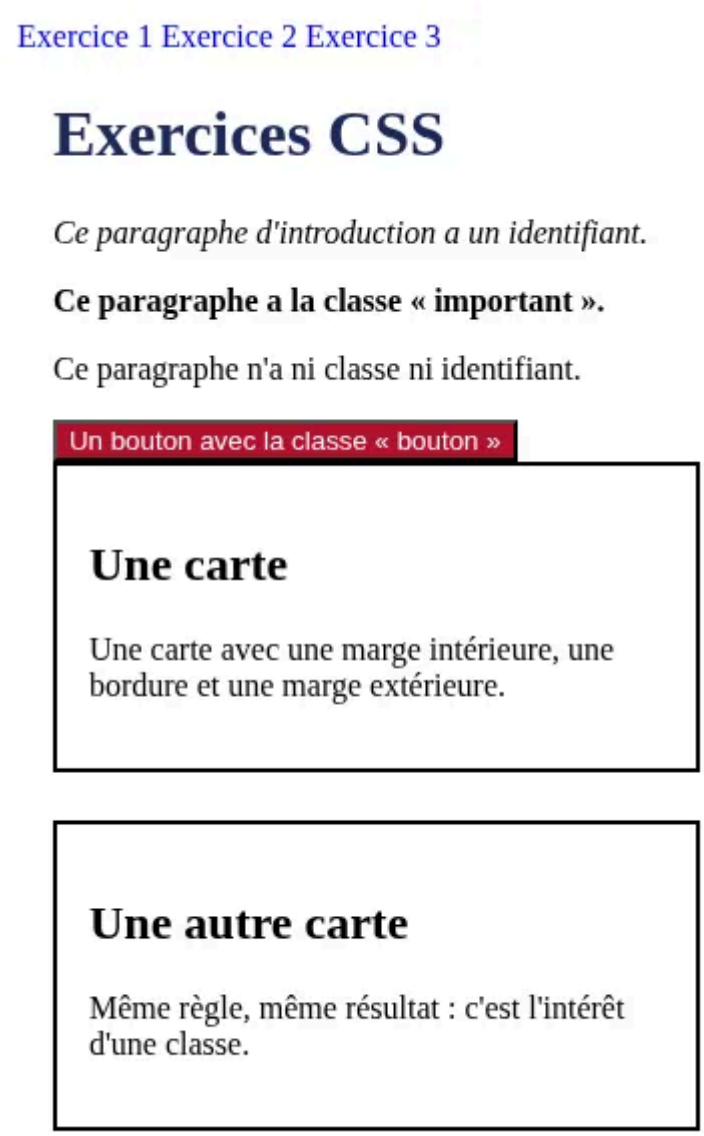
Exercice 3 — Les cartes (15 min)

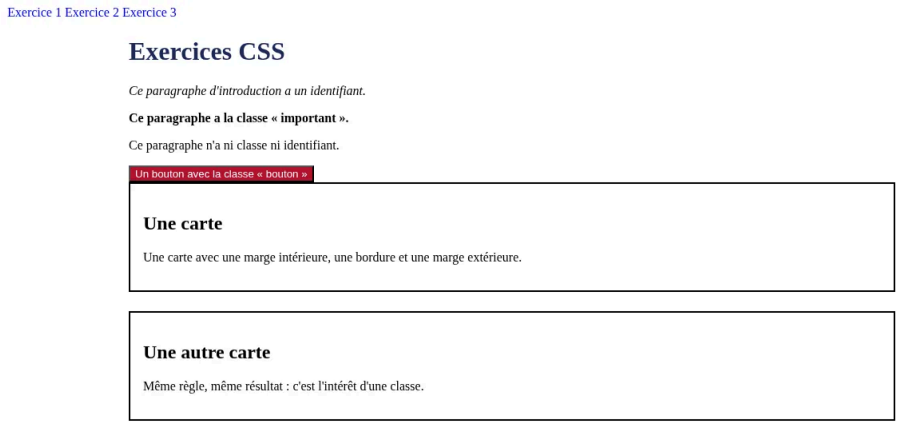
1. Seuls : **exercice 3**, `box-sizing` et les cartes (`.carte`).
2. **Réussi si** : les tests des exercices 1 à 3 passent.

- L’exercice 4 : mercredi matin.

Niveau : guidé.

Résultat attendu (la solution complète, exercice 4 de mercredi compris, à 375 px et en largeur bureau ; cliquez pour agrandir) :





Enregistrer et envoyer (2 min)
Fin du TP : dans le terminal, depuis `~/dwwm/dwwm-tp/semaine-05` :

```
git status
git add css-exercices/style.css
git commit -m "Ajoute les cartes de l'exercice 3"
git push
```

Réussi si : `git status` affiche `Votre branche est à jour avec 'origin/main'` et `rien à valider, la copie de travail est propre` ; sur GitHub, le commit apparaît dans `dwwm-tp`. Le message suit la règle de la semaine 4 : ce que fait le commit, à l'impératif, en une ligne (fiche aide-mémoire commit du Drive partagé, ou fiche de révision).

Les outils du navigateur (15 min)

1. Inspectez une carte de vos exercices (clic droit, *Inspecter*).
2. Décochez son `padding`, puis recochez-le.
3. Onglet *Calculé* : lisez son box model.
4. **Réussi si** : vous retrouvez les valeurs de l'exercice 3.

Niveau : guidé.

Sur vos exercices CSS terminés (exercice 3 compris), dans Chrome (Edge est identique). Ce que vous changez dans les outils n'est jamais enregistré.

1. Ouvrez `css-exercices/index.html` dans Chrome, faites un clic droit sur une carte, puis *Inspecter*. **Vous devez voir** : le panneau *Éléments*, la ligne `<article class="carte">` surlignée, et la carte colorée dans la page.
2. Dans l'onglet *Styles*, survolez la déclaration `padding` de la règle `.carte`, puis décochez sa case. **Vous devez voir** : la déclaration barrée, et la carte qui perd sa marge intérieure ; recochez la case : tout revient.
3. Ouvrez l'onglet *Calculé* (*Computed*). **Vous devez voir** : en haut, le schéma du box model, quatre rectangles emboîtés (`margin`, `border`, `padding`, contenu) avec leurs valeurs ; survolez une couche : la zone se colore dans la page.

4. Pour jeudi : inspectez un élément en `display: flex` ou `grid` (le menu de votre page, ou une page de ce site). **Vous devez voir** : dans le panneau *Éléments*, un petit badge `flex` ou `grid` au bout de la ligne ; un clic sur le badge dessine la mise en page par-dessus la page (Google, s. d.-c, s. d.-e).

À retenir — ce matin

> À retenir
>
> - **Règle** : sélecteur, puis `propriété: valeur;`.
> - **Cascade** : `id` > classe > élément ; à égalité, la dernière gagne.
> - **Box model** : contenu, `padding`, `border`, `margin` ; `border-box`.

Le détail, notion par notion :

- **Règle et sélecteurs** : `p`, `.classe`, `#id`, `parent enfant` ; un point-virgule oublié fait ignorer la déclaration suivante.
- **Cascade** : la spécificité d'abord, puis l'ordre ; F12 montre la règle gagnante et barre les autres.
- **Box model** : quatre couches ; `box-sizing: border-box` en haut de chaque feuille de style ; l'onglet *Calculé* les montre.

Débrief

- Pourquoi `.bouton` gagne-t-il, avant comme après `button` ?
- Que change `box-sizing: border-box` ?
- Que montre l'onglet *Calculé* des outils du navigateur ?

Après-midi — TP — CSS Diner, puis « CSS from scratch »

Apport 5 — Une interface propre (10 min)

- **Hiérarchie** : ce qui compte le plus se voit en premier (taille, graisse).
- **Échelle d'espacements** : 4 ou 5 valeurs, pas une de plus.
- **Typographie** : 3 ou 4 tailles, interligne 1,5.
- **Palette limitée** : texte, fond, une couleur d'accent pour l'action.
- **Alignement** : des bords communs, texte à gauche.

Pourquoi. Une interface « propre » n'est pas une question de talent : c'est l'application de quelques règles, toujours les mêmes. Un visiteur trouve plus vite ce qu'il cherche.

Définition. Cinq règles : la hiérarchie, une échelle d'espacements, une typographie limitée, une palette limitée, l'alignement. Pour le texte, les références s'accordent : un corps de 15 à 25 px à l'écran, un interligne de 120 à 145 % de la taille du texte, des lignes de 45 à 90 caractères (Butterick, s. d.) ; web.dev conseille une largeur maximale en `ch` (45 à 75 caractères) et un `line-height` sans unité (web.dev (Google Chrome), s. d.-b). Le texte centré se lit mal au-delà de deux lignes : alignez à gauche. Pourquoi des éléments proches semblent-ils « aller ensemble » ? Les lois de la perception (proximité, similarité) l'expliquent : voir *Laws of UX* (Yablonski, s. d.).

Avant / après, sur une petite carte (pas celle de l'exercice) :

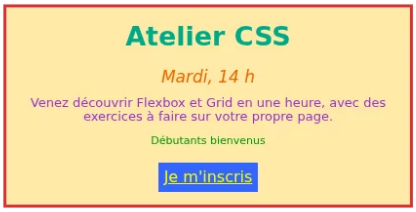


Fig. 1. – Avant : cinq tailles, six couleurs, tout centré



Fig. 2. – Après : une hiérarchie, trois couleurs, aligné à gauche

Pas à pas. Le formateur projette la page brouillonne de l'exercice ; pour chaque règle, le groupe montre ce qui ne va pas (trop de tailles, couleurs au hasard, texte centré…).

Interface propre — attention et vérification

> ⚠ Attention
>
> Des espacements « presque » dans l'échelle (`0.6rem` au lieu de `0.5rem`) : la page paraît brouillonne sans qu'on sache pourquoi. Prenez toujours une valeur de l'échelle.

- **Vérifiez** : combien de couleurs d'accent dans une palette limitée ?

Apport 6 — Les variables CSS (5 min)

- **Pourquoi** : changer une couleur partout, à un seul endroit.
- **Variable CSS** : déclarée dans `:root`, utilisée avec `var(--…)`.

Pourquoi. Si la couleur d'accent est écrite dans vingt règles, la changer demande vingt corrections, et on en oublie une.

Définition. Une variable CSS (propriété personnalisée) se déclare une fois dans `:root` et s'utilise partout avec `var()` (MDN Web Docs, s. d.-e). Les tailles des exemples sont en `rem` (1rem vaut 16 px par défaut) : recopiez les valeurs de l'échelle telles quelles ; l'unité `rem` est expliquée mercredi matin.

Exemple.

```
:root {
  --couleur-accent: #0b6e4f;
  --espace-3: 1rem;
  --texte-titre: 2rem;
}
h1 { font-size: var(--texte-titre); }
.bouton { background: var(--couleur-accent); }
```

Avant / après : une seule ligne changée dans `:root` (`--couleur-accent`, de `#0b6e4f` à `#b3122e`) ; le titre et les deux boutons, qui l'utilisent, changent ensemble.



Fig. 3. – Avant : –couleur-accent: #0b6e4f

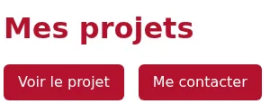


Fig. 4. – Après : –couleur-accent: #b3122e

Pas à pas.

1. Déclarez `--couleur-accent` dans `:root`.
2. Utilisez-la dans la règle `.bouton` avec `var(--couleur-accent)`.

3. Changez la valeur dans `:root`, enregistrez : tous les boutons changent.

Variables — attention et vérification

> ⚠ Attention
>
> `var(--couleur-acent)` (mal écrit) ne donne aucune erreur : la propriété est simplement ignorée. Vérifiez l'orthographe, au caractère près.

- **Vérifiez** : où déclare-t-on les variables de toute la page ?

Exercice — Avant / après (30 min)

1. Dossier `interface-propre` : une page volontairement brouillonne.
2. Faites une capture **avant** (375 px et grand écran).
3. Réécrivez `style.css` : les cinq règles (consignes en commentaire).
4. **Réussi si** : `interface-propre.test.js` passe ; capture **après**.

> 💡 Défi
>
> Les règles appliquées strictement : aucune couleur écrite à la main hors de `:root`, aucun espacement hors de l'échelle, un interligne entre 1,4 et 1,8. **Réussi si** : `node --test interface-propre.defi.test.js` passe.

Niveau : semi-guidé.

Les cinq tests vérifient une règle chacun (les trois couleurs et l'échelle d'espacements dans `:root`, chaque `font-size` dans l'échelle, le titre et le bouton, une colonne alignée à gauche) ; vos yeux vérifient le résultat. Mettez les deux captures côte à côte : qu'est-ce qui a le plus changé ?

Enregistrer (1 min)
Dans le terminal, depuis `~/dwwm/dwwm-tp/semaine-05` :

```
git status
git add interface-propre/style.css
```

```
git commit -m "Applique les cinq règles à la page brouillonne"
```

Réussi si : `git status` affiche `rien à valider, la copie de travail est propre`.

« CSS from scratch » (1 h 05)

1. Stylez votre page personnelle, dans `page-perso/style.css`.
2. Commencez par `:root` : votre palette, vos espacements, vos tailles.
3. Appliquez les cinq règles d'une interface propre.
4. Contrastes : 4,5:1 au moins pour le texte (pas de colonnes : jeudi).
5. **Réussi si** : `node --test page-perso-style.test.js` passe.

- Pas fini ? « Si le temps le permet », puis jeudi après-midi.

Niveau : semi-guidé.

Vérifiez chaque couple de couleurs (texte et fond) avec l'outil de contraste du navigateur, en suivant le tutoriel ci-dessous : le niveau AA des WCAG demande un rapport d'au moins 4,5:1 pour le texte normal et 3:1 pour le grand texte (W3C, s. d.). Choisissez deux ou trois couleurs au plus, rangées dans `:root`, et gardez-les partout. Le test `page-perso-style.test.js` vérifie que vos couleurs et vos espacements sont des variables.

Tutoriel — Mesurer un contraste dans le navigateur (10 min)
Sur la page brouillonne de l'exercice, dans Chrome (Edge est identique) (Google, s. d.-d).

1. Dans Chrome, ouvrez `interface-propre/index.html`, la version **brouillonne** du kit (Live Preview : *Ouvrir dans le navigateur*). **Vous devez voir** : la page « Réparez votre vélo vous-même », aux couleurs vives, avec un pied de page gris clair.
2. Faites un clic droit sur le pied de page (« Maison de quartier · 12 rue des Lilas · Entrée libre »), puis *Inspecter*. **Vous devez voir** : les outils du navigateur s'ouvrent sur le panneau *Éléments* ; la ligne `<p>` du pied de page est surlignée.

3. Dans l'onglet *Styles*, trouvez la règle `footer` et sa déclaration `color: #95a5a6` ; cliquez sur le petit carré de couleur placé juste avant `#95a5a6`. **Vous devez voir** : le sélecteur de couleur s'ouvre : une grande zone de couleur, des curseurs, la valeur.
4. Sous les curseurs, lisez la ligne du rapport de contraste (*Contrast ratio*). **Vous devez voir** : un nombre proche de 2,4, suivi d'un symbole d'alerte : le gris clair sur fond presque blanc n'est pas assez lisible. Si la ligne affiche un score « APCA » au lieu d'un rapport comme `2.45`, ouvrez les paramètres des outils (roue dentée), rubrique *Préférences*, section *Éléments*, et décochez la case *APCA* ; rouvrez le sélecteur.
5. Cliquez sur cette ligne pour la déplier. **Vous devez voir** : les seuils **AA** (4,5) et **AAA** (7), et deux lignes tracées dans la zone de couleur : au-dessous de la ligne AA, le texte devient lisible.
6. Appuyez sur Échap. Ce que vous changez ici n'est pas enregistré : corrigez la couleur dans `style.css`, puis mesurez de nouveau.

Firefox : même chemin (clic droit, *Examiner*, puis le carré de couleur dans les règles) ; le sélecteur de couleur indique si le contraste avec le fond respecte les recommandations, et le survol du message donne le détail (Mozilla, s. d.-d). Les noms des menus peuvent être traduits dans votre navigateur.

Lisible ou pas ? Six couples de couleurs, avec leur rapport de contraste calculé par la formule des WCAG :



Figure 6. – `#1f2a5a` sur `#ffffff` : 13,66:1, lisible



Figure 7. – `#767676` sur `#ffffff` : 4,54:1, lisible, tout juste



Figure 8. – `#999999` sur `#ffffff` : 2,84:1, pas lisible



Figure 9. – `#ffffff` sur `#b3122e` : 6,90:1, lisible

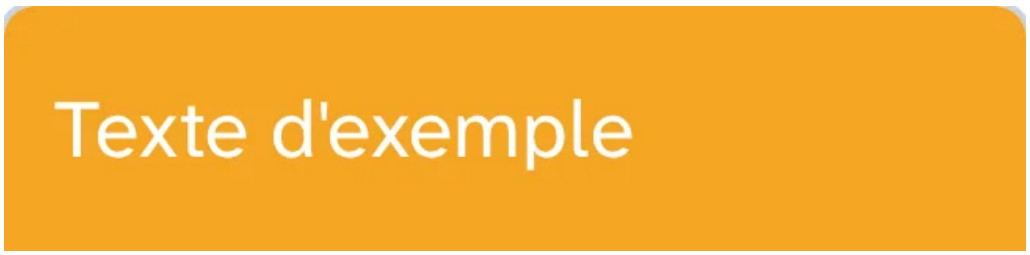


Figure 10. – `#ffffff` sur `#f5a623` : 2,02:1, pas lisible

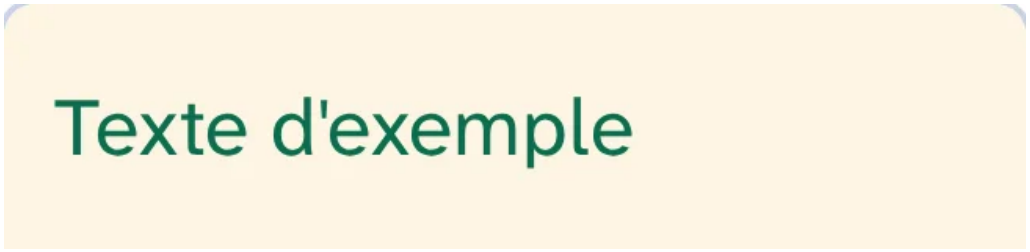


Figure 11. – `#0b6e4f` sur `#fdf6e3` : 5,79:1, lisible

> **Note**
>
> **Enregistrer et envoyer (2 min)**
> Fin du TP : dans le terminal, depuis `~/dwwm/dwwm-tp/semaine-05` :
>
> ```
> git status
> git add page-perso/style.css
> git commit -m "Ajoute le style de la page personnelle"
> git push
> ```
>
> **Réussi si** : `git status` affiche `Votre branche est à jour avec 'origin/main'` et `rien à valider, la copie de travail est propre` ; sur GitHub, le commit apparaît dans `dwwm-tp`. Le message suit la règle de la semaine 4 : ce que fait le commit, à l'impératif, en une ligne (fiche aide-mémoire commit du Drive partagé, ou fiche de révision).

Les éditeurs du navigateur

Les outils de développement (F12) de Chrome et de Firefox cachent de petits éditeurs visuels dans le panneau des styles : cliquez sur l'icône placée à côté d'une valeur.

- **Sélecteur de couleur** : la pastille à côté de chaque couleur ; il affiche aussi le rapport de contraste du texte avec son fond (Google, s. d.-d; Mozilla, s. d.-d).
- **Éditeur d'ombres** (Chrome) : à côté de `box-shadow` et de `text-shadow` (Google, s. d.-b).
- **Éditeur de courbes d'animation** : à côté d'une fonction de temporisation (`ease`, `cubic-bezier()`) (Google, s. d.-b; Mozilla, s. d.-e).
- **Simulation des troubles de la vision** : la page vue sans le rouge, sans le vert, sans le bleu, sans couleur ou avec moins de contraste ; une information portée par la seule couleur s'y perd. Dans Chrome : dans les outils, `Cmd+Shift+P` (mac), `Ctrl+Shift+P` (windows), tapez `rendering`, puis réglez *Emulate vision deficiencies* (Google, s. d.-f, s. d.-a) ; dans Firefox : panneau *Accessibilité*, menu *Simuler* (Mozilla, s. d.-a). Les noms des menus peuvent être traduits dans votre [illegible] (Google, s. d.-c; Mozilla, s. d.-b, s. d.-c).
- **Inspecteurs flexbox et grid** : les badges `flex` et `grid` dessinent la mise en page sur l'écran ; ils serviront jeudi à vérifier ce que vous modifiez dans `style.css`, en

Échec à éviter : passer à l'étape suivante sans avoir vérifié le résultat dans le navigateur.

À retenir — cet après-midi

À retenir

- **Interface propre** : hiérarchie, échelle, typographie, palette, alignement.
- **Variables** : déclarées dans `:root`, utilisées avec `var(--…)`.
- **Éditeurs du navigateur** : le CSS à la main d'abord, l'éditeur pour ajuster.

Le détail, notion par notion :

- **Interface propre** : cinq règles, toujours les mêmes ; une échelle d'espacements et de tailles, une palette de trois couleurs, du texte aligné à gauche.
- **Variables CSS** : un choix, un seul endroit (`:root`) ; ailleurs, jamais de valeur en dur ; une faute de frappe dans le nom ne donne aucune erreur.

Critères de réussite

- ☐ `css-exercices` : les tests des exercices 1 à 3 au vert
- ☐ `interface-propre` au vert ; captures avant / après
- ☐ Page perso : `page-perso-style` au vert, contrastes vérifiés

Si le temps le permet — La page perso, plus loin

- Terminez le style de votre page : chaque section, chaque carte.

Défi

- Stylez `:hover` **et** `:focus-visible` sur les liens.
- Testez au clavier : le focus doit toujours être visible.

Pour aller plus loin

- CSS Diner : les sélecteurs, en jeu (remédiation de mercredi).
- Utopia : une échelle de tailles et d'espacements qui s'adapte à l'écran.
- Lecture de référence : *Learn CSS*, sur web.dev.
- Compatibilité : le tableau en bas de chaque page MDN.

Une propriété est-elle reconnue par tous les navigateurs ? Le tableau *Browser compatibility*, en bas de chaque page de

référence de MDN, l'indique navigateur par navigateur ; ses données sont publiques et maintenues par MDN (MDN contributors, s. d.). Regardez-le pour `gap` ou `aspect-ratio`.

CSS Diner (flukeout, s. d.) est passé au niveau 1 du parcours de remédiation de mercredi. Utopia propose des calculateurs gratuits d'échelles typographiques et d'espacements fluides (Gilyead et Mudford, s. d.) : c'est une découverte, l'échelle fixe de cet après-midi suffit. *Learn CSS*, sur web.dev, reprend le CSS chapitre par chapitre (boîtes, sélecteurs, cascade, spécificité, couleurs, unités, mise en page) (web.dev (Google Chrome), s. d.-a). La traduction française est automatique : en cas de doute, la version anglaise (bouton de langue en haut de la page) fait foi. Lecture libre, sans temps obligatoire.

Débrief

- Quelle règle de l'interface propre a le plus changé votre page ?
- Pourquoi ranger les couleurs dans des variables ?

Sources

Butterick, Matthew. s. d. « Practical Typography: Summary of key rules ». Consulté le 1 octobre 2026. https://practicaltypography.com/summary-of-key-rules.html.

flukeout. s. d. « CSS Diner - Where we feast on CSS Selectors! » Consulté le 26 septembre 2026. https://flukeout.github.io/.

Gilyead, James, et Trys Mudford. s. d. « Utopia: Fluid responsive design ». Consulté le 1 octobre 2026. https://utopia.fyi/.

Google. s. d.-a. « Apply other effects: enable automatic dark theme, emulate focus, and more — Chrome DevTools ». Consulté le 3 octobre 2026. https://developer.chrome.com/docs/devtools/rendering/apply-effects.

Google. s. d.-b. « CSS features reference — Chrome DevTools ». Consulté le 3 octobre 2026. https://developer.chrome.com/docs/devtools/css/reference.

Google. s. d.-c. « Inspect and debug CSS flexbox layouts — Chrome DevTools ». Consulté le 3 octobre 2026. https://developer.chrome.com/docs/devtools/css/flexbox.

Google. s. d.-d. « Inspect and debug HD and non-HD colors with the Color Picker — Chrome DevTools ». Consulté le 3 octobre 2026. https://developer.chrome.com/docs/devtools/css/color.

Google. s. d.-e. « Inspect CSS grid layouts — Chrome DevTools ». Consulté le 3 octobre 2026. https://developer.chrome.com/docs/devtools/css/grid.

Google. s. d.-f. « Rendering tab overview — Chrome DevTools ». Consulté le 3 octobre 2026. https://developer.chrome.com/docs/devtools/rendering.

MDN contributors. s. d. « Browser compatibility tables and Browser Compatibility Data (BCD) — MDN Web Docs ». Consulté le 3 octobre 2026. https://developer.mozilla.org/en-US/docs/MDN/Writing_guidelines/Page_structures/Compatibility_tables.

MDN Web Docs. s. d.-a. « Gérer les conflits - Apprendre ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Learn_web_development/Core/Styling_basics/Handling_conflicts.

MDN Web Docs. s. d.-b. « Le modèle de boîte - Apprendre ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Learn_web_development/Core/Styling_basics/Box_model.

MDN Web Docs. s. d.-c. « Sélecteurs CSS de base - Apprendre ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Learn_web_development/Core/Styling_basics/Basic_selectors.

MDN Web Docs. s. d.-d. « Spécificité - CSS ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/CSS/Guides/Cascade/Specificity.

MDN Web Docs. s. d.-e. « Utiliser des propriétés personnalisées CSS (variables) ». Consulté le 1 octobre 2026. https://developer.mozilla.org/fr/docs/Web/CSS/Using_CSS_custom_properties.

MDN Web Docs. s. d.-f. « Valeurs et unités CSS - Apprendre ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Learn_web_development/Core/Styling_basics/Values_and_units.

Mozilla. s. d.-a. « Color vision simulation — Firefox Source Docs ». Consulté le 3 octobre 2026. https://firefox-source-docs.mozilla.org/devtools-user/accessibility_inspector/simulation/.

Mozilla. s. d.-b. « CSS Flexbox Inspector: Examine Flexbox layouts — Firefox Source Docs ». Consulté le 3 octobre 2026. https://firefox-source-docs.mozilla.org/devtools-user/page_inspector/how_to/examine_flexbox_layouts/.

Mozilla. s. d.-c. « CSS Grid Inspector: Examine grid layouts — Firefox Source Docs ». Consulté le 3 octobre 2026. https://firefox-source-docs.mozilla.org/devtools-user/page_inspector/how_to/examine_grid_layouts/.

Mozilla. s. d.-d. « Inspect and select colors — Firefox Source Docs ». Consulté le 3 octobre 2026. https://firefox-source-docs.mozilla.org/devtools-user/page_inspector/how_to/inspect_and_select_colors/.

Mozilla. s. d.-e. « Work with animations — Firefox Source Docs ». Consulté le 3 octobre 2026. https://firefox-source-docs.mozilla.org/devtools-user/page_inspector/how_to/work_with_animations/.

W3C. s. d. « Understanding Success Criterion 1.4.3: Contrast (Minimum) ». Consulté le 28 septembre 2026. https://www.w3.org/WAI/WCAG22/Understanding/contrast-minimum.html.

web.dev (Google Chrome). s. d.-a. « Learn CSS (traduction automatique en français ; textes sous licence CC BY 4.0) ». Consulté le 2 octobre 2026. https://web.dev/learn/css/box-model?hl=fr.

web.dev (Google Chrome). s. d.-b. « Learn Responsive Design: Typography ». Consulté le 1 octobre 2026. https://web.dev/learn/design/typography.

Yablonski, Jon. s. d. « Laws of UX ». Consulté le 1 octobre 2026. https://lawsofux.com/.